Analytics Engineering: Session 3

Sources, Seeds, Dependencies & First Tests

Agenda

- Configuring Sources
- Working with Seeds
- Creating Staging Models
- Source Freshness
- Introduction to Generic Tests
- The `dbt build` Command

Configuring Sources

Sources represent raw data loaded into your warehouse by an EL tool (or `create_db.py` in our case).

Define them in a `sources.yml` file under `models/`:

```yaml
version: 2

sources:
  - name: raw
    tables:
      - name: customers
        description: "Raw customer data"
```

Reference them in SQL: `{{ source('raw', 'customers') }}`

Creating Staging Models from Sources (1)

Staging models clean and prepare raw source data for further transformations.

```
with source_data as (
    select * from {{ source('raw', 'customers') }}
)

select * from source_data
where email is not null
```

This creates a `stg_customers` model that downstream models can reference. Then, we can create a table in the database by running `dbt run`.

Creating Staging Models from Sources (2)

When writing the `from {}` clause for reading from a source, always use the `source()` function. It has 2 arguments:

1. Source name (as defined in `sources.yml`), in our case `raw`.
2. Table name (as defined in `sources.yml`), e.g., `customers`, `orders`, etc.

Staging Models: Do's and Don'ts

What to do in Staging:

- **Renaming**: Rename columns to be descriptive and consistent (snake_case).
- **Casting**: Ensure correct data types (strings to dates, etc.).
- **Basic Cleaning**: Handle nulls, trim strings.
- **1:1 Mapping**: One staging model per source table.

What NOT to do in Staging:

- **Joins**: Do not join with other tables (save for Intermediate).
- **Aggregations**: No `GROUP BY` (save for Marts).
- **Business Logic**: Keep it raw-ish.

Working with Seeds

Seeds are CSV files in your `seeds/` directory. Best for static data that changes infrequently.

1. Add `segments.csv` to `seeds/`.
2. Run `dbt seed`.
3. Reference in SQL: `{{ ref('segments') }}`.

Do not use seeds for large raw data!

Source Freshness

Check if source data is up to date.

```yaml
sources:
  - name: raw
    tables:
      - name: orders
        loaded_at_field: loaded_at
        freshness:
          warn_after: {count: 12, period: hour}
          error_after: {count: 24, period: hour}
```

Run with: `dbt source freshness`

This catches broken pipelines before they impact downstream models.

Using the Codegen Package

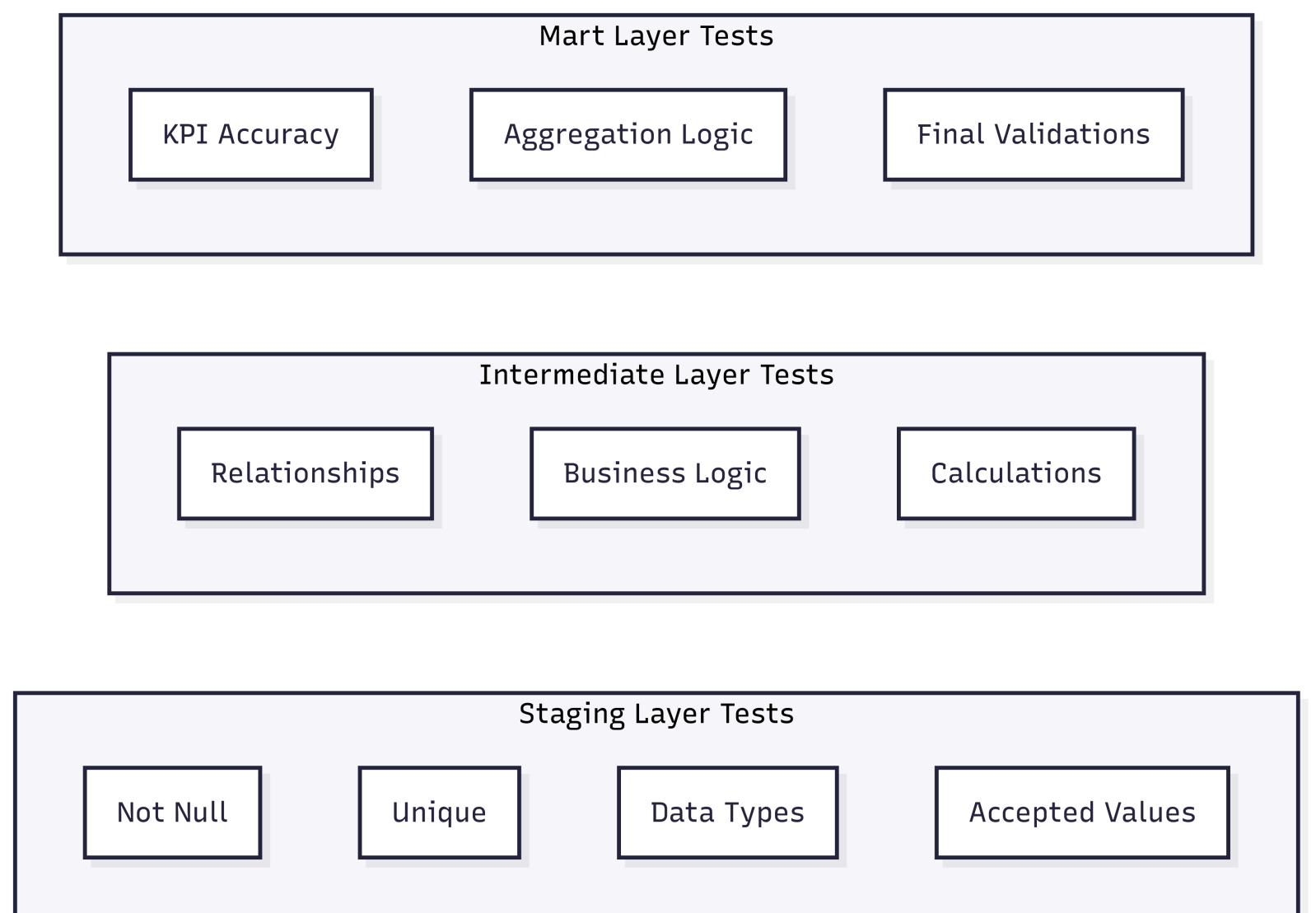
The `codegen` package generates boilerplate YAML and SQL for you.

```
-- Generate a sources.yml for all tables in a schema
{{ codegen.generate_source('main', database_name='my_database') }}

-- Generate a staging model for a source table
{{ codegen.generate_base_model(
    source_name='raw',
    table_name='customers'
) }}
```

Run with: `dbt compile` then check `target/compiled/` for the output.

Introduction to Generic Tests



Introduction to Generic Tests

Tests validate assumptions about your data. dbt has 4 built-in **generic tests**:

Test	What it checks
`unique`	No duplicate values in a column
`not_null`	No null values in a column
`accepted_values`	Values are from a known set
`relationships`	Foreign key integrity

We'll start with `unique` and `not_null` — the essential primary key tests.

Adding Tests to Staging Models

Define tests in a YAML file alongside your models:

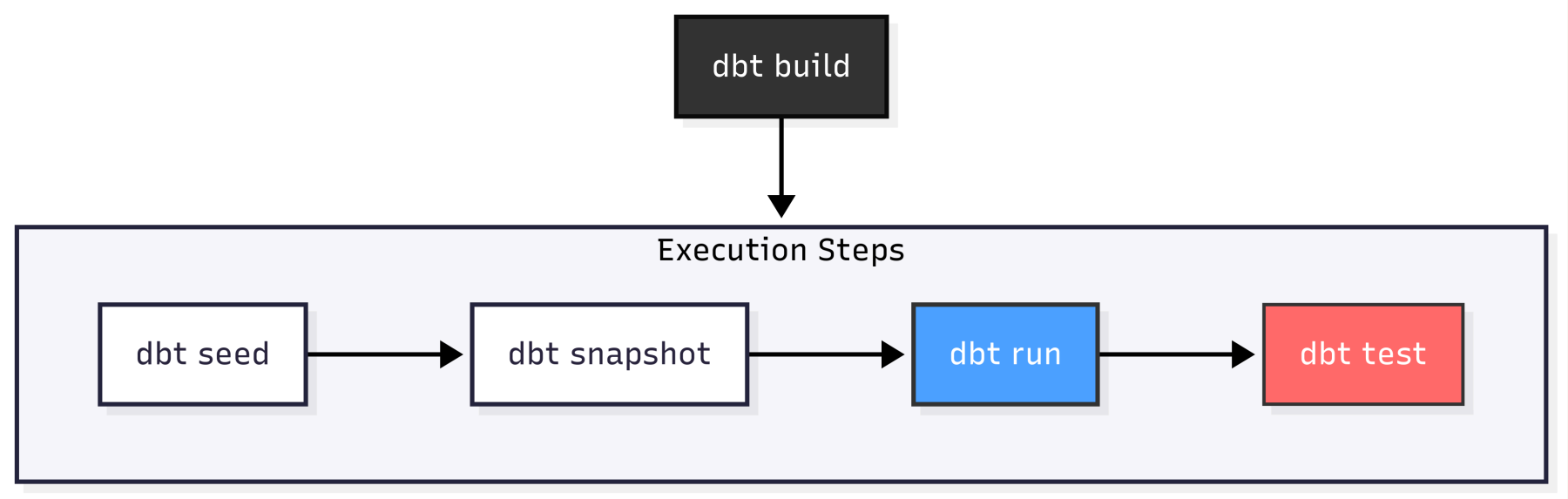
```yaml
version: 2

models:
  - name: stg_customers
    columns:
      - name: customer_id
        description: "Primary key"
        tests:
          - unique
          - not_null
```

Run with: `dbt test -s stg_customers`

If the test returns any rows, it fails — meaning your assumption is violated.

The `dbt build` Command



The `dbt build` Command

`dbt build` runs **models, tests, seeds, and snapshots** together in DAG order.

```
dbt build                # Build everything
dbt build -s staging     # Build only staging layer
```

Why use `dbt build` instead of `dbt run`?

- Models are built, then immediately tested.
- If a test fails, downstream models **won't run**.
- This prevents bad data from propagating.

From now on, prefer `dbt build` over `dbt run` in your workflow.

Inspecting Compiled SQL

Understand what dbt sends to the database.

1. **Compile**: Click the "Compile" button (or run `dbt compile`).
2. **View SQL**: Check `target/compiled/.../stg_customers.sql`.
   - Notice how `{{ source(...) }}` is replaced by `"my_database"."main"."customers"`.

This is critical for debugging logic errors!

Visualizing Lineage

See how data flows through your project.

1. **Lineage Tab**: Click the "Lineage" button in the extension bar.
2. **Graph**:
   - **Left**: `source.raw.customers` (The Parquet file/Table).
   - **Right**: `stg_customers` (Your model).
3. **Impact**: If you change the source, you see immediately what breaks.

What have we learned in this session

- Define sources in `sources.yml`
- Load static data with seeds
- Create staging models from raw sources
- Check source freshness with `dbt source freshness`
- Add `unique` and `not_null` tests to staging models
- Use `dbt build` to run models and tests together

Next Session: Data Modeling & Modularity.

Work!

- Define sources in `sources.yml` for every raw table in the database.
- Create staging models for every source table.
- Add a seed file `segments.csv` that maps customer segments: `customer_segment,segment_id`
- Add `unique` and `not_null` tests to all staging model primary keys.
- Run `dbt build` and verify all models and tests pass.